

Plan Assurance Qualité

T-ESP-800

Naavi

Eloi Jahan

Alexandre Tressel

Charlotte Laurent

Julien Mercourt

Raphaël Eouzan

Valentin Ragot

Yann Plouhinec

Table des matières

1. Introduction 4
1.1. Objectif du document 4
1.2. Présentation du projet 5
2. Gestion de projet 7
2.2. Processus d'offboarding 8
2.3. Charte de mentorat 9
2.4. Procédure de post-mortem (blameless review) 9
2.5. Organisation de l'équipe 10
3. Méthodologie de travail 11
3.1. Méthode de gestion de projet 11
3.2. Organisation des jalons 11
3.3. Work Breakdown Structure (WBS) 12
3.4. Gestion des livraisons 12
4. Spécification technique 16
4.1. Analyse des besoins 16
4.2. Conception visuelle 17
4.2.5. Documentation et livrables 18
4.3. Architecture 19
4.4. DevOps 21
4.5. Normes et bonnes pratiques 25
4.6. Pratiques de sécurité 27
4.7. Normes de contribution Git 27
4.8. Conception fonctionnelle 29
4.8.3. Fonctionnalités de l'application mobile 32
4.8.4. Fonctionnalités du back-office 33
4.9. Phase de test 34
5. Business Plan 35
5.1. Objectifs 35
5.2. Parties prenantes 36
5.3. Business model 39

5.4. Stratégie de marketing 39
5.5. Plan de communication 42

1. Introduction

1.1. Objectif du document

Ce Plan d'Assurance Qualité (PAQ) est conçu pour définir et documenter les processus et les normes mis en place afin de garantir la qualité du projet, de sa conception à sa livraison finale. L'objectif principal est de s'assurer que tous les livrables répondent aux exigences techniques et fonctionnelles, tout en satisfaisant les attentes des utilisateurs et des collaborateurs impliqués.

Le document commence par une introduction qui présente les objectifs du PAQ et l'importance de la qualité dans le cadre du projet. Cette section offre une vue d'ensemble du projet, de ses objectifs globaux et de son contexte, permettant ainsi à tous les intervenants de comprendre les enjeux et les attentes associés à ce projet.

La gestion de projet est ensuite abordée en détail, en expliquant l'organisation de l'équipe, les rôles et les responsabilités de chaque membre. La méthodologie de travail utilisée pour la planification, le suivi et la gestion des risques est également décrite, afin de garantir une approche structurée et cohérente tout au long du projet. Les jalons du projet sont clairement définis, avec une description des étapes clés et des livrables attendus à chaque phase. La gestion des livraisons est également couverte, en expliquant les processus de livraison et de validation des livrables, assurant ainsi que chaque étape est réalisée conformément aux normes de qualité établies.

La méthodologie de travail est un autre aspect crucial abordé dans ce document. Elle inclut la description des méthodes de gestion de projet, l'organisation des jalons, et la gestion des livraisons. Les collaborateurs, qu'ils soient internes ou externes, sont identifiés, et leurs rôles dans le projet sont clairement définis. Cette section vise à garantir que tous les intervenants comprennent leur contribution au succès du projet et les attentes en matière de qualité.

La spécification technique est une partie essentielle de ce PAQ, couvrant divers aspects techniques du projet. L'analyse des besoins est détaillée, expliquant la méthodologie utilisée pour recueillir et analyser les besoins des utilisateurs et des parties prenantes.

La conception visuelle est également abordée, avec une description des principes de design et des outils utilisés pour créer une interface utilisateur intuitive et efficace. L'architecture technique du projet est présentée, incluant les choix technologiques et les principes de conception qui sous-tendent le développement. Les pratiques DevOps mises en place pour assurer une intégration et un déploiement continus sont expliquées, garantissant ainsi une livraison rapide et fiable des fonctionnalités. Les normes et bonnes pratiques suivies pour garantir la qualité du projet sont listées, offrant une référence claire pour tous les intervenants. La conception fonctionnelle est

également décrite, détaillant les fonctionnalités du projet et les processus de conception.

Enfin, la phase de test est présentée, avec une explication des stratégies de test mises en place pour valider les livrables et s'assurer qu'ils répondent aux exigences de qualité.

Le business plan est une autre composante importante de ce document. Il définit les objectifs commerciaux du projet et identifie les parties prenantes clés et leurs attentes. Le modèle économique du projet est décrit, offrant une vue d'ensemble de la manière dont le projet générera de la valeur. La stratégie de marketing pour promouvoir le projet est également présentée, garantissant que les efforts de communication et de promotion sont alignés avec les objectifs du projet. Enfin, les sources de financement et la gestion budgétaire du projet sont expliquées, assurant ainsi une gestion financière rigoureuse et transparente.

En suivant ce PAQ, nous nous engageons à maintenir un haut niveau de qualité tout au long du projet, en assurant que chaque étape est réalisée conformément aux normes et aux meilleures pratiques de l'industrie. Ce document servira de référence pour tous les intervenants du projet, garantissant ainsi une compréhension commune des objectifs de qualité et des processus à suivre pour les atteindre.

1.2. Présentation du projet

1.2.1. Contexte

La randonnée, activité en forte croissance, reste aujourd'hui confrontée à des enjeux majeurs de sécurité et de partage d'information. Chaque année, de trop nombreux incidents et accidents, souvent évitables, surviennent sur les sentiers faute d'alertes en temps réel ou de communication efficace entre randonneurs. Par ailleurs, les solutions grand public disponibles proposent généralement un accès gratuit très limité l'utilisateur se trouve rapidement bloqué dès qu'il souhaite explorer de nouveaux territoires ou obtenir des données à jour.

Le manque de "sociabilité" numérique nuit à la cohésion et à l'entraide entre passionnés : peu d'applications permettent de signaler et de consulter, au sein d'une même communauté, les dangers de la journée, les points de ravitaillement ou les zones hors sentier. Les offices de tourisme et fédérations locales peinent à diffuser leurs données et actualités directement auprès des utilisateurs terrain, ce qui crée une difficulté entre les informations "officielles" et l'expérience vécue des randonneurs.

Naavi prend racine dans ce constat : offrir une plateforme mobile (Android et iOS) alliant sécurité, partage communautaire et accès libre à des itinéraires fiables. Grâce à un système de signalement collaboratif des dangers, à des cartes interactives hors-ligne et à un modèle "à la carte", Naavi vise à réduire significativement les accidents

évitables tout en renforçant les liens entre randonneurs et acteurs locaux (offices de tourisme, fédérations, ...).

1.2.2. Objectifs du projet

1. Sécurité des randonneurs
   Mettre en place un système de signalement collaboratif des dangers (chutes, zones impraticables, conditions météo...) et d'alertes en temps réel, afin de réduire le nombre d'incidents évitables sur les sentiers.

2. Fiabilité et disponibilité
   Assurer une disponibilité de l'application ≥ 99 % et un temps de réponse des API 200 ms, grâce à une architecture microservices scalable et à un déploiement Blue-Green.

3. Expérience utilisateur
   Proposer une interface intuitive et accessible (WCAG 2.1 AA), avec mode hors-ligne complète des cartes et filtres de recherche (difficulté, distance, dénivelé), pour que 90 % des utilisateurs jugent l'app "facile à prendre en main" lors du premier usage.

4. Couverture fonctionnelle
   Livraison incrémentale de la V1 à V3 : du parcours basique et suivi de position (V1), à la gamification et partage social (V2), puis au mode collaboratif de création de points d'intérêt éphémères et notifications d'alerte (V3).

5. Qualité logicielle
   Atteindre ≥ 70 % de couverture de tests unitaires et d'intégration, appliquer systématiquement les normes ISO 25010 et ISO 29119, et maintenir un indice de qualité Trivy > 8.0.

6. Modèle économique et engagement
   Lancer une application avec des pubs, l'utilisateur pourrait ne plus les avoir en payant un abonnement.

7. Communauté et partenariats
   Créer une communauté de bêta-testeurs (Discord/Instagram) d'au moins 1 000 membres, et établir au moins 5 partenariats avec offices de tourisme ou fédérations locales d'ici 6 mois.

8. Amélioration continue
   Mettre en place un backlog produit priorisé, des rétrospectives sprints

systématiques et un suivi des KPIs (taux de crash, NPS, temps moyen de session) pour ajuster la roadmap en continu.

2. Gestion de projet

2.1 Processus d'onboarding

Objectif : Intégrer un nouveau collaborateur de façon rapide et efficace, en assurant la montée en compétence sur l'ensemble des outils et bonnes pratiques du projet.

Étapes détaillées et formations :

1. **Avant l'arrivée** :
    a. Préparation des comptes (Git, GitLab CI/CD, Figma)
    b. Envoi d'un kit de bienvenue : documentation projet, charte de contribution, planning des formations
    c. Désignation d'un mentor référent
2. **Jour 1 – Découverte de l'environnement** :
    a. **Session 1 : Présentation globale du projet** (objectifs, périmètre, architecture)
    b. **Session 2 : Installation et configuration** des environnements de développement (IDE, SDK Flutter, Java)
3. **Semaine 1 – Formations techniques et processus** :
    a. **Formation Git & GitLab** : workflow GitLab Flow, branches, merge requests, résolution de conflits
    b. **Formation CI/CD** : pipelines, qualité de code (Trivy), déploiement Blue-Green
    c. **Formation Jira & gestion de tickets** : création/gestion des US, bugs, tâches, sprints
    d. **Atelier bonnes pratiques** : revue de code, tests unitaires & E2E, normes ISO applicables
4. **Semaine 2 – Prise en main du code et du cycle Agile** :
    a. **Code walkthrough** : découverte du code mobile (Flutter) et backend (Spring Boot)
    b. **Sprint simulé** : participation à un mini-sprint pour appliquer les rituels (planning, daily, review)
5. **Premier mois – Validation et suivi** :
    a. Réalisation d'une première petite US avec revue par le mentor
    b. Point de suivi à J+30 pour ajuster le plan de montée en compétence

Checklist d'onboarding

Tâche	Responsable	Statut
Création des comptes Git, Jira, CI/CD	Lead Dev / Chef de projet	☐
Installation des outils de développement	Lead Dev	☐
Présentation de l'équipe	Chef de projet	☐
Présentation de l'architecture	Lead Tech	☐
Première revue de code	Lead Tech	☐
Bilan à 1 mois	Chef de projet	☐

2.2. Processus d'offboarding

Objectif

Assurer une transition sécurisée et documentée lors du départ d'un collaborateur.

Étapes détaillées

1. **Notification et planification**
    a. Réception de la demande de départ ou licenciement.
    b. Communication à l'équipe
2. **Transfert de connaissances**
    a. Mise à jour de la documentation existante par le travail du collaborateur.
    b. Session de transfert avec collègues clés.
3. **Désactivation des accès**
    a. Révocation des droits sur les dépôts git, outils et serveurs.
4. **Entretien de sortie**
    a. Recueil du feedback du collaborateur avec son mentor
5. **Archivage**
    a. Sauvegarde des livrables, configurations.
    b. Clôture des tickets Jira.

Checklist d'offboarding

Tâche	Responsable	Statut

Notification à toute l'équipe	Mentor / Chef de projet	☐
Transfert de connaissances réalisé	Lead Tech / Lead Dev / Collaborateur	☐
Désactivation des accès	Lead Tech	☐
Entretien de sortie	Mentor	☐
Archivage des artefacts	Lead Tech	☐

2.3. Charte de mentorat

Objectif

Formaliser la relation mentor/mentoré pour accélérer la montée en compétence.

Élément	Description
Rôle du mentor	Soutien technique, accompagnement des premières tâches, feedback
Rôle du mentoré	Engagement, préparation des points hebdomadaires
Objectifs	Liste des compétences visées et livrables attendus
Fréquence des points	Hebdomadaire (30-60 min) en présentiel ou distanciel avec le mentoré
Indicateurs de succès	Réussite de tâches, autonomie croissante

2.4. Procédure de post-mortem (blameless review)

Objectif

Analyser les incidents pour en tirer des enseignements et améliorer continuellement les process sans recherche de coupable.

Étapes

1. **Préparation**
    a. Collecte des logs, tickets, dates et faits.
    b. Concertation avec toute l'équipe
2. **Session de revue**
    a. Reconstitution de la chronologie.
    b. Brainstorming des causes sous-jacentes.
    c. Proposition de mesures correctives.

3. **Rédaction du rapport**
    a. Contexte, chronologie, causes identifiées, actions et responsables (gitlab issues)
4. **Suivi**
    a. Mise en place des actions.
    b. Vérification de l'efficacité dans le sprint suivant.

2.5. Organisation de l'équipe

2.3.1. Organigramme de l'équipe (OBS)

- **Chef de projet** : [Eloi Jahan] – Supervision générale
- **Responsable Qualité** : [Eloi Jahan, Yann Plouhinec] – Vérification des normes et tests
- **Développeurs mobiles** : [Charlotte Laurent, Eloi Jahan, Yann Plouhinec, Personne recruté] – Développement et intégration
- **Développeurs back-office** : [Alexandre Tressel, Personne recruté]
- **DevOps :** [Valentin Ragot, Alexandre Tressel] – Gestion des versions et des outils

2.3.2. Rôles et responsabilités

- **Chef de projet** : Suivi du projet, gestion des risques, reporting.
- **Développeurs** : Implémentation des fonctionnalités.
- **Testeurs** : Conception et exécution des tests.
- **Responsable Qualité** : Vérification du respect des standards de qualité.
- **Responsable Configuration** : Gestion des livrables et versions.

2.3.3 Processus de gestion des problèmes :

1. **Identification** : tout membre peut ouvrir un ticket GitLab étiqueté incident ou bug
2. **Classification** :
    a. **Criticité P1** : blocage production (SLA 4h)
    b. **Criticité P2** : impact fonctionnel majeur (SLA 1 jour)
    c. **Criticité P3** : demande d'amélioration / bug mineur (SLA 3 jours)
3. **Escalade** :

    a. P1 : avertir immédiatement Lead Tech et Chef de projet, réunion d'urgence
    b. P2 : assignation au Lead Dev, point de suivi quotidien
    c. P3 : tri et planification dans backlog sprint suivant
4. **Résolution** :
    a. Analyse de la cause racine (post-mortem si P1/P2)
    b. Développement du correctif, tests unitaires et intégration
    c. Validation en préproduction
    d. Déploiement via pipeline CI/CD
5. **Suivi et retour d'expérience** :
    a. Mise à jour de la documentation (wiki interne)
    b. Rétrospective spécifique pour incident P1/P2

3. Méthodologie de travail

3.1. Méthode de gestion de projet

Le projet adopte une méthodologie **Scrum** (Agile), qui permet une gestion itérative et incrémentale du développement. Cette approche facilite l'adaptation aux changements et garantit une livraison progressive des fonctionnalités.

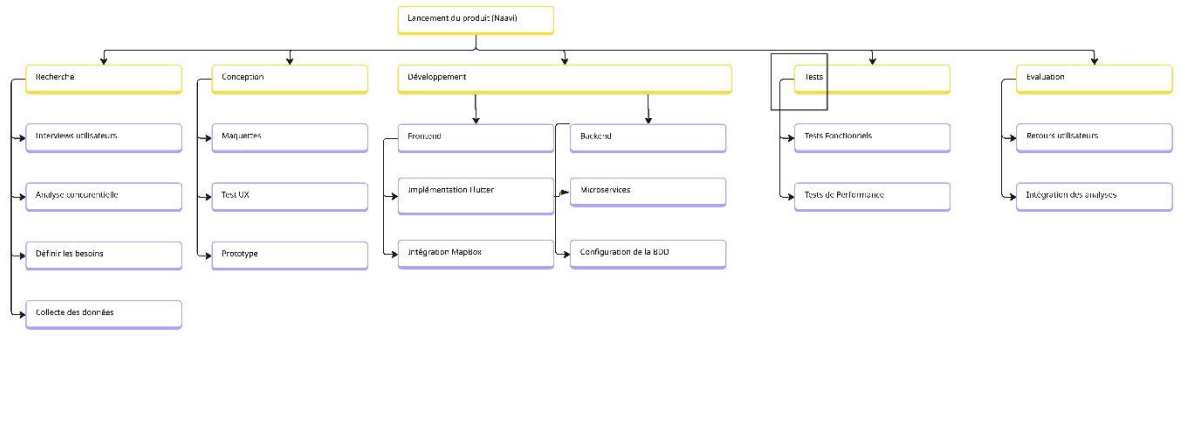
3.2. Organisation des jalons

Les étapes clés du projet sont les suivantes :

- **Sprint Planning** : Planification des tâches, définition des objectifs du sprint, priorisation des tickets issus du backlog produit (créés via GitLab).
- **Développement** : Implémentation des fonctionnalités selon les user stories sélectionnées. Chaque tâche est suivie via un système de tickets pour assurer la traçabilité et la répartition du travail.
- **Revue de Sprint** : Démonstration des fonctionnalités terminées, présentation des livrables, retour des parties prenantes.
- **Rétrospective** : Discussion en équipe sur le déroulé du sprint, identification des points de blocage, mise en place d'actions correctives pour les sprints suivants.

3.3. Work Breakdown Structure (WBS)

Dans le cadre de notre projet nous avons élaboré un **Work Breakdown Structure (WBS)**. Ce découpage hiérarchique du projet permet de structurer les activités en sous-tâches cohérentes, permettant de fluidifier la planification notre projet.



Dans le cas de **Naavi**, le WBS est organisé autour des grandes étapes du cycle de vie du produit :

- La recherche (interviews utilisateurs, analyse concurrentielle)
- La conception (maquettes, tests UX),
- Le développement (frontend avec Flutter, backend en Java),
- Les tests (fonctionnels et de performance),
- Ainsi que l'évaluation continue à travers les retours utilisateurs.

Ce découpage reflète l'approche agile adoptée, en permettant une gestion claire des livrables à chaque sprint et une visibilité partagée sur la progression globale du projet.

3.4. Gestion des livraisons

3.4.1. Processus de livraison

Le processus de livraison est conçu pour garantir une mise en production fluide, stable et sécurisée de l'application. Il suit une approche progressive qui permet de valider chaque modification avant sa mise à disposition des utilisateurs finaux. Cette approche repose sur quatre étapes clés : le développement et les tests, la validation interne, la livraison sur les stores, ainsi que le support post-livraison.

3.4.1.1. Phase de développement

La première phase concerne le développement et les tests, où chaque fonctionnalité est implémentée conformément aux spécifications fonctionnelles et techniques définies en amont. L'implémentation suit les principes du développement

agile, avec des cycles courts permettant d'assurer une amélioration continue du produit. Durant cette phase, les développeurs suivent des pratiques rigoureuses et l'intégration continue, ce qui permet de détecter rapidement les anomalies. Les tests unitaires sont exécutés pour chaque composant afin d'assurer que chaque module fonctionne correctement de manière isolée.

3.4.1.2. Phase de test d'intégration

Ensuite, des tests d'intégration sont réalisés pour valider la bonne communication entre les micro-services (Authentification, Géo, Randonnées, Abonnement, Notification). Des outils comme JUnit, Postman sont utilisés pour automatiser ces tests et garantir une couverture optimale (70% de couverture minimum). Une revue de code systématique est également effectuée avant toute fusion de code afin d'assurer un respect strict des bonnes pratiques et des normes de développement (ex. conventions de codage, design patterns).

3.4.1.3. Validation interne

Une fois la phase de développement finalisée, la seconde phase consiste en la validation interne. L'application est déployée sur un environnement de préproduction avec Firebase, qui reproduit les conditions réelles d'utilisation sans impacter la production. Cet environnement permet aux testeurs et au Chef de Projet, responsable de la validation finale, d'effectuer des tests approfondis. Ces tests incluent :

- Tests fonctionnels pour vérifier que les nouvelles fonctionnalités répondent bien aux attentes.
- Tests de régression pour s'assurer qu'aucune modification ne provoque de dysfonctionnements ailleurs dans l'application.
- Tests de charge et de performance, notamment grâce à Prometheus, afin d'évaluer la capacité de l'application à supporter un nombre croissant d'utilisateurs.

Si des anomalies critiques sont détectées, elles doivent être corrigées avant toute mise en production. Ce processus de validation est crucial pour minimiser les risques et garantir un produit final robuste et fiable.

3.4.1.4. Publication sur les stores

Une fois la validation interne effectuée avec succès, l'application passe en phase de livraison sur les stores. Cette étape consiste à publier une nouvelle version de l'application sur les plateformes officielles. Pour Google Play, l'application est générée sous format APK et soumise aux contrôles de Google. Pour l'App Store, un IPA est soumis à Apple, qui effectue une vérification approfondie avant validation. Chaque nouvelle version est accompagnée d'un changelog détaillé, permettant aux utilisateurs de prendre connaissance des ajouts et des améliorations apportées.

3.4.1.5. Documentation (peut-être à décaler ou re facto)

En parallèle, la documentation technique est mise à jour, notamment Swagger pour l'API de chaque micro-services, afin de garantir que les développeurs disposent toujours d'une référence à jour. Une mise à jour de la documentation interne est également effectuée si nécessaire, notamment pour faciliter l'intégration de nouveaux membres dans l'équipe.

Le support post-livraison est un élément essentiel de la gestion des livraisons, car il assure un suivi réactif des incidents et une assistance aux utilisateurs. Un formulaire intégré à l'application permet aux utilisateurs de signaler des bugs ou des problèmes techniques. Ce formulaire est directement relié à GitLab via l'API, permettant une création automatique de tickets, qui sont ensuite assignés aux équipes de développement. Les bugs critiques sont traités en priorité et font l'objet d'un hotfix immédiat, déployé sous forme de patch (PATCH versioning) afin de limiter l'impact sur les utilisateurs. Les correctifs moins urgents sont intégrés dans le sprint suivant, afin de garantir une gestion efficace des ressources et de respecter la planification initiale.

Enfin, la maintenance évolutive est assurée par la gestion d'un backlog produit, qui centralise toutes les demandes d'amélioration et permet de prioriser les développements futurs. Un système de support client est également mis en place sous forme de FAQ, regroupant les réponses aux questions les plus courantes et permettant aux utilisateurs de trouver rapidement des solutions aux problèmes rencontrés.

3.4.2. Planification des livraisons

3.4.2.1. Sémantique

Le versioning appliqué suit le schéma MAJOR.MINOR.PATCH. Une version MAJOR est incrémentée lorsqu'une modification majeure est introduite, pouvant impliquer une incompatibilité avec les versions précédentes. Une version MINOR correspond à l'ajout de nouvelles fonctionnalités sans impact sur l'existant, tandis qu'une version PATCH est utilisée pour corriger des bugs ou améliorer les performances sans modifier le comportement de l'application.

3.4.2.2. Déploiement progressif

Le déploiement progressif est assuré grâce à la méthode Blue-Green Deployment. Cette approche repose sur l'existence de deux environnements distincts : une version "bleue" actuellement en production et une version "verte" contenant la nouvelle mise à jour. Une fois la version verte testée et validée, un basculement est effectué sans interruption de service pour l'utilisateur. Ce mécanisme permet d'assurer un rollback rapide en cas de problème et de réduire considérablement les risques liés à la mise en production.

3.4.2.3. Monitoring

Afin de garantir un suivi optimal de la qualité et des performances, l'application est équipée d'un système de monitoring avancé avec Prometheus. Cet outil permet de collecter et d'analyser des métriques en temps réel, offrant une visibilité complète sur l'état du système. Il facilite la détection proactive des anomalies et permet d'anticiper d'éventuels problèmes de charge ou de stabilité. Par ailleurs, un système de logs persistants est mis en place, permettant d'assurer une traçabilité complète des erreurs et facilitant leur résolution rapide.

3.4.2.4. Amélioration continue

Enfin, l'analyse des retours utilisateurs constitue un élément central de l'amélioration continue de l'application. Des enquêtes intégrées permettent de recueillir les avis des utilisateurs sur leur expérience et d'identifier les points d'amélioration prioritaires. Les avis laissés sur les stores (Google Play et App Store) sont également analysés régulièrement afin d'adapter la roadmap du produit en fonction des attentes et des besoins exprimés par la communauté.

4. Spécification technique

4.1. Analyse des besoins

L'application Naavi est conçue pour répondre aux besoins des randonneurs, qu'ils soient débutants ou expérimentés. Les utilisateurs recherchent une solution intuitive et complète pour planifier, suivre et partager leurs randonnées. Ils souhaitent également bénéficier de fonctionnalités avancées telles que la consultation des performances et le suivi d'autres usagers.

En tant qu'utilisateur de l'application Naavi, je souhaite pouvoir planifier mes randonnées en créant et personnalisant mes itinéraires. Je veux également être renseigner des différents points d'intérêt pour choisir mon parcours et consulter les randonnées disponibles en appliquant des filtres comme la difficulté, la durée ou la localisation.

En tant qu'utilisateur de l'application Naavi, je souhaite disposer d'une carte interactive pour m'orienter lors de mes randonnées. Je veux pouvoir suivre ma position en temps réel et utiliser un mode hors-ligne pour continuer ma navigation même sans connexion Internet.

En tant qu'utilisateur de l'application Naavi, je souhaite avoir un profil personnel que je peux créer et gérer. Je veux y retrouver mon historique de randonnées ainsi que mes statistiques de performance. Un système de récompenses doit être mis en place pour me motiver à utiliser régulièrement l'application.

En tant qu'utilisateur de l'application Naavi, je souhaiterais pouvoir interagir avec mes amis en les suivant et en consultant leurs performances. Je veux aussi partager mes randonnées et mes points d'intérêt avec eux.

En tant qu'utilisateur de l'application Naavi, je souhaite recevoir des notifications pour être informé des événements et des mises à jour importantes.

En tant qu'utilisateur de l'application Naavi, je souhaite avoir la possibilité de signaler un danger ou un incident lors de mes randonnées. Je veux également accéder à des informations de sécurité et à des conseils pratiques pour garantir ma protection.

En tant qu'utilisateur de l'application Naavi, je souhaite bénéficier d'une application performante et fiable. Elle doit me permettre de naviguer de manière fluide et garantir la stabilité des fonctionnalités, même en cas de faible connectivité.

4.2. Conception visuelle

La conception visuelle de l'application repose sur une approche UX/UI centrée sur l'utilisateur, visant à assurer une expérience fluide et intuitive. Pour garantir une cohérence graphique et faciliter la collaboration entre les équipes, nous utilisons Figma comme principal outil de conception.

4.2.1. Outils et méthodologie

- Outil principal : Figma
- Collaboration : Travail en équipe avec des commentaires en temps réel et des itérations rapides.
- Prototypage interactif : Création de maquettes dynamiques pour simuler les parcours utilisateurs avant le développement.

4.2.2. Conception mobile

L'application mobile est designer pour offrir une interface moderne et accessible :

- Guidelines : Respect des recommandations Flutter.
- Typographie et couleurs : Définition d'une charte graphique adaptée aux besoins des utilisateurs.
- Parcours utilisateur : Conception de wireframes pour modéliser l'expérience utilisateur avant l'UI finale.

4.2.3. Conception back-office

L'interface du back-office est pensée pour une utilisation efficace et optimisée sur desktop :

- UI/UX : Design sobre et fonctionnel, adapté aux administrateurs et gestionnaires.
- Optimisation : Prise en compte des principes responsive pour assurer un affichage correct sur différents écrans.

4.2.4. Charte graphique

La charte graphique définit l'identité visuelle de l'application et garantit une cohérence entre les différentes interfaces (mobile et back-office). Elle inclut les éléments suivants :

4.2.4.1 Palette de couleurs

L'application suit une palette de couleurs harmonisée, définie pour assurer une expérience utilisateur agréable et accessible :

- Couleur primaire : Utilisée pour les éléments interactifs (boutons, liens, accents).

- Couleur secondaire : Complémentaire à la couleur primaire, utilisée pour les éléments d'accentuation.
- Couleurs neutres : Définition des couleurs de fond, des textes et des séparateurs.
- Feedbacks et états : Couleurs spécifiques pour les erreurs, avertissements et succès.

4.2.4.2 Typographie

La charte graphique impose une typographie claire et lisible sur tous les écrans :

- Police principale : [Nova round] – utilisée pour les titres et textes principaux.
- Police secondaire : [Lexend deca] – utilisée pour les sous-titres et éléments secondaires.
- Hiérarchie des textes : Définition des tailles et épaisseurs ("Title", "Title2", "Body1", "Body1Bold", "Body2", "Body2Bold", "Body2Light", "Body3", Body3Light").

4.2.4.3 Iconographie et pictogrammes

L'application utilise une bibliothèque d'icônes cohérente pour améliorer la navigation et l'ergonomie :

- Bibliothèque d'icônes : bibliothèque libre de droit trouvée sur Figma Community
- Style : Icônes en outline ou en filled, avec un design homogène.
- Utilisation : Icônes utilisées pour les actions, catégories et états du système.

4.2.4.4 Composants UI et design system

Un design system est mis en place pour assurer une uniformité graphique et fonctionnelle :

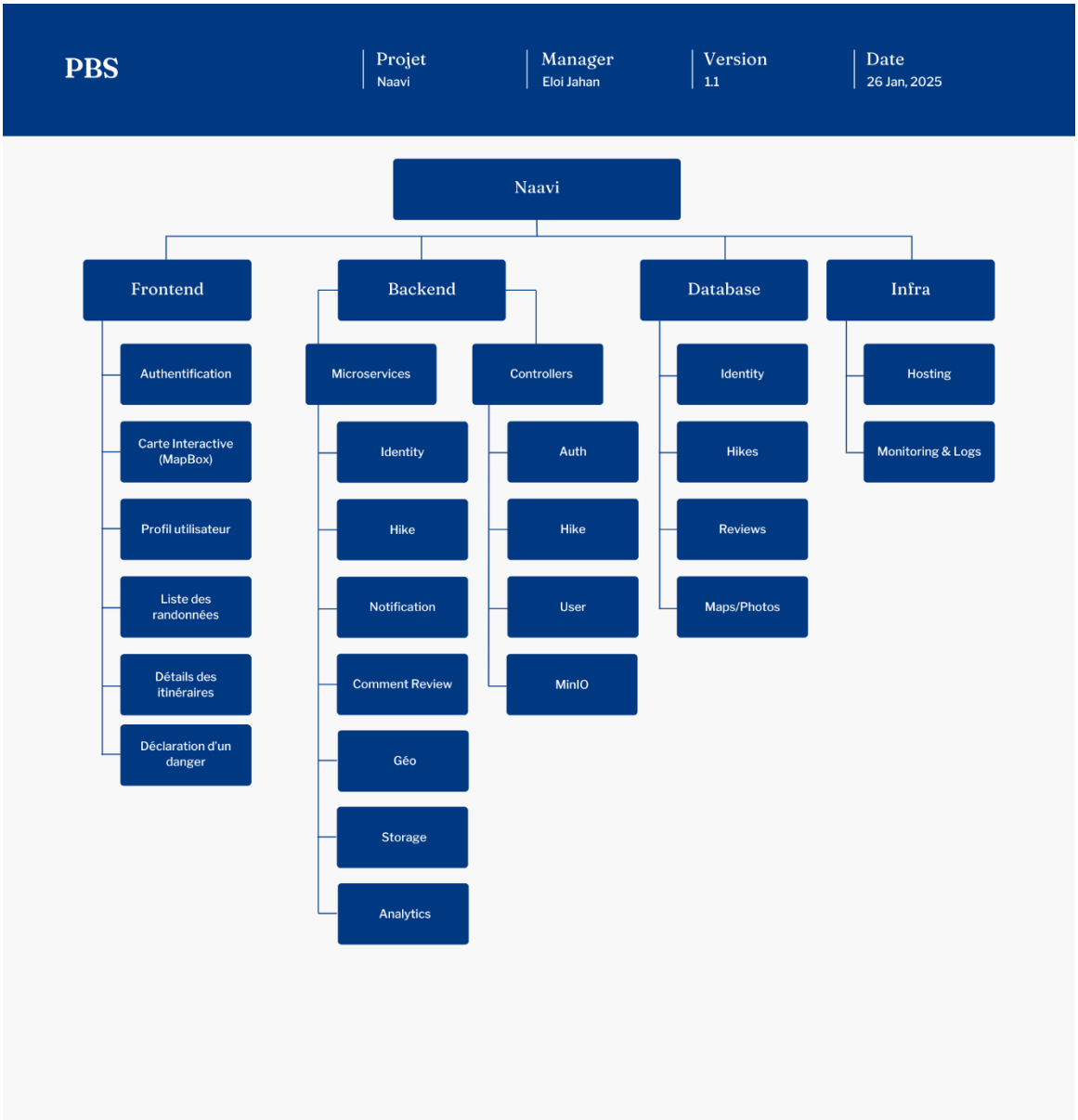
- Boutons : Styles définis pour les boutons activés et désactivés.
- Formulaires : Styles des champs de saisie, labels, messages d'erreur et validation.
- Cartes et modales : Apparence des éléments contenant des informations et des interactions.
- Animations et transitions : Définition des effets pour améliorer l'expérience utilisateur par des commentaires.

4.2.5. Documentation et livrables

À chaque étape du projet, les maquettes et prototypes seront mis à jour et validés en collaboration avec l'équipe technique et les parties prenantes. Les livrables incluent :

- Wireframes basse fidélité : Esquisses des parcours utilisateurs et des interfaces.

- Maquettes haute-fidélités : Design finalisé avant l'intégration.
- Prototypes interactifs : Simulation des interactions principales avant le développement.



4.3. Architecture

L'architecture de l'application repose sur une approche modulaire et scalable, intégrant des technologies éprouvées pour garantir la robustesse, la maintenabilité et la sécurité de l'ensemble du système. Elle est divisée en trois principales couches : application mobile, back-office, et backend, avec une base de données relationnelle centralisée et un stockage des médias distribué.

4.3.1. Application mobile

L’application mobile est développée avec Flutter en respectant les principes de la Clean Architecture, permettant une séparation claire des responsabilités en trois couches :

- Presentation Layer : Interface utilisateur (UI) et gestion des interactions utilisateur.
- Domain Layer : Logique métier, cas d'utilisation et entités.
- Data Layer : Gestion des accès aux données et aux API.

Les principales technologies utilisées sont :

- Langage : Dart
- Framework : Flutter
- Base de données : PostgreSQL (via le backend)
- Stockage des médias : MinIO (compatible S3)
- Cartographie : Mapbox API

L’application communique avec le backend via des API REST sécurisées, et des mécanismes d’authentification basés sur JWT assurent la gestion des sessions utilisateur.

4.3.2. Back-end

Le backend est conçu selon une architecture microservices, permettant une meilleure scalabilité et une indépendance fonctionnelle des différents services métiers.

Les principales technologies utilisées sont :

- Langage : Java
- Framework : Spring Boot
- Architecture : Microservices
- Base de données : PostgreSQL
- Stockage des médias : MinIO (compatible S3)
- Sécurité : Spring Security avec JWT

Chaque micro-service gère une fonctionnalité spécifique et communique avec les autres via des API REST sécurisées.

Communication de sécurité :

- API REST : Communication entre le front-end (mobile et web) et le backend via des API sécurisées.
- Authentification & Autorisation : Gestion des utilisateurs via JWT, avec un contrôle d’accès basé sur les rôles.

4.4. DevOps

La mise en œuvre d'une démarche DevOps est au cœur du cycle de vie du projet **Naavi**. Elle vise à automatiser l'intégralité des processus de développement, test, livraison et déploiement afin de garantir une intégration rapide, fiable et itérative des nouvelles fonctionnalités tout en assurant la qualité, la sécurité, la traçabilité et la maintenabilité du code.

4.4.1. Outils et infrastructure

Le projet repose principalement sur **GitLab** (Ultimate) comme plateforme centralisée de gestion du code, CI/CD, gestion des tickets et sécurité. Les runners GitLab hébergés sont utilisés pour l'exécution des pipelines. L'infrastructure est conçue pour supporter des déploiements évolutifs via **Kubernetes**, avec un monitoring temps réel via **Prometheus** et **Grafana**, et un stockage cloud via **Scaleway S3**.

4.4.2. Organisation CI/CD

Chaque dépôt de microservice ou application (mobile Flutter ou backoffice Angular) utilise un fichier `.gitlab-ci.yml` qui orchestre les étapes CI/CD suivantes :

- **Build** : Compilation des artefacts, image Docker pour le backend, APK/IPA pour Flutter.
- **Analyse de qualité** : Analyse statique via **Trivy**, vérification des duplications, complexité, vulnérabilités et couverture de test.
- **Tests automatisés** :
    - **Tests unitaires** et **d'intégration** pour le backend Java (JUnit).
    - **Tests de widget/unitaires Flutter** côté mobile.
    - **Tests de composants Angular** pour le backoffice.
- **Scan sécurité** : Analyse des dépendances via **Trivy**, détection des vulnérabilités OWASP.
- **Déploiement** :
    - Environnement de **préproduction** automatisé.
    - Déploiement **blue/green** sur Kubernetes pour une bascule sans interruption.
    - Déploiement mobile en **bêta test** via Firebase App Distribution.

4.4.3. Templates CI/CD partagés

Afin de ne pas dupliquer les étapes CI dans chaque dépôt, un **repository central de templates CI/CD** est mis en place. Chaque projet importe dynamiquement ces templates via `include:` dans son `.gitlab-ci.yml`.

Cela permet :

- Une **mutualisation des pipelines** pour l'ensemble des services (build, lint, test, deploy, trivy, semgrep...).
- Une **mise à jour centralisée** des bonnes pratiques DevOps.
- Une **personnalisation locale** via des variables GitLab et des jobs `extends`.

4.4.4. Stratégie de déploiement

Le backend, packagé sous forme de containers Docker, est déployé sur un cluster Kubernetes avec :

- Des services exposés via **Ingress NGINX**.
- Un **circuit de déploiement blue/green** pour rollback rapide.
- Un **auto-scaling horizontal** basé sur les métriques CPU/RAM.

Les images sont versionnées (`MAJOR.MINOR.PATCH`) et hébergées dans le registre GitLab.

4.4.5. Monitoring, alerting et logs

Chaque service dispose :

- De **métriques Prometheus** (temps de réponse, disponibilité, erreurs HTTP).
- De **logs centralisés** accessibles via un système ELK ou Kibana.
- De **dashboards Grafana** pour le suivi de l'expérience utilisateur et des performances.
- De **systèmes d'alertes** automatisés en cas de dérives de SLA (latence > 200ms, erreurs > 5%).

4.4.6. Sécurité DevSecOps

L'approche DevOps intègre des pratiques **DevSecOps** :

- Analyse automatique des vulnérabilités de code et dépendances (GitLab Security Dashboard, SAST/DAST).

- Gestion des secrets via des **variables d'environnement protégées**.
- Politique d'authentification forte (OAuth2.0, JWT), MFA pour les membres du projet.
- Revue de code obligatoire avec **validation de pipeline CI/CD** avant tout merge sur `develop` ou `main`.

4.4.7. Environnements et delivery mobile

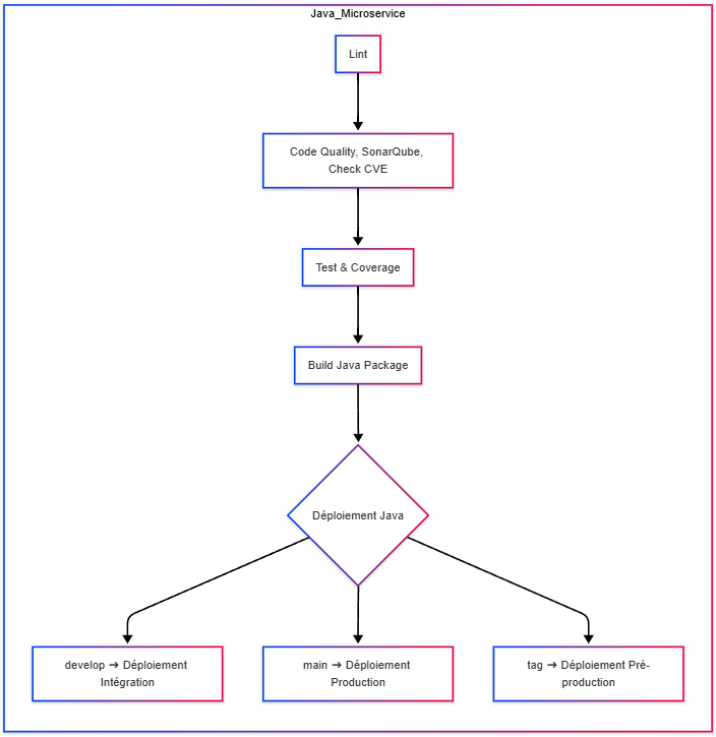
Environnement	Objectif	Accès	Publication
dev	Tests de fonctionnalité	Équipe	Interne
preprod	Tests fonctionnels/QA	PO	Interne
prod	Utilisateurs finaux	Public	Google Play / App Store

Les versions Flutter sont signées automatiquement lors des pipelines, puis livrées :

- En bêta via Firebase.
- En production via **Google Play Console** et **Apple App Store Connect**, avec versioning et changelog automatisés.




Frontend_Angular
Lint
Code Quality, SonarQube, Check CVE
Test & Coverage
Build Angular Package
Déploiement
develop → Déploiement Intégration
main → Déploiement Production
tag → Déploiement Pré-production


Java_Microservice
Lint
Code Quality, SonarQube, Check CVE
Test & Coverage
Build Java Package
Déploiement Java
develop → Déploiement Intégration
main → Déploiement Production
tag → Déploiement Pré-production

4.5. Normes et bonnes pratiques

4.5.1. ISO 9001 – Gestion de la Qualité

Cette norme assure un processus de développement optimisé en mettant en place un système de gestion de la qualité efficace. Cela implique :

- Un suivi rigoureux des processus de développement.
- Une amélioration continue des méthodes employées.

4.5.2. ISO 25010 – Qualité des Logiciels et des Systèmes

Garantit la fiabilité, la maintenabilité et la performance de l'application. Elle impose des critères de qualité tels que :

- La correction des erreurs et bogues.
- L'optimisation du code source.
- La facilité d'adaptation et d'extension.

4.5.3. ISO/IEC 27001 – Sécurité des Systèmes d'Information

Pour garantir la sécurité des données, un Système de Gestion de la Sécurité de l'Information (SGSI) doit être mis en place :

- Authentification forte (MFA, OAuth2.0, JWT).
- Chiffrement des données sensibles.
- Surveillance et gestion des vulnérabilités (mises à jour, audits réguliers).

4.5.4. ISO/IEC 27701 – Protection des Données Personnelles

Extension de la norme ISO 27001, elle assure :

- La gestion de la confidentialité des données utilisateurs.
- La conformité avec le RGPD et autres réglementations.
- La mise en place d'un registre des traitements des données.

4.5.5. ISO/IEC 27017 – Sécurité des Services Cloud

Cette norme est recommandée si le projet utilise un hébergement cloud pour le stockage des données :

- Renforcement de la sécurité des infrastructures cloud.
- Contrôle des accès et journalisation des activités.

4.5.6. ISO 9241 – Ergonomie des Interactions Homme-Machine (IHM)

L'interface utilisateur doit respecter les bonnes pratiques UX/UI :

- Des boutons larges et clairs avec des labels explicites.
- Une navigation intuitive et sans surcharge d'informations.
- Un mode sombre et un mode contrasté pour l'accessibilité.

- Des tests utilisateurs réguliers pour adapter l'expérience aux besoins des randonneurs.

4.5.7. ISO 30071-1 – Accessibilité Numérique

Pour garantir une accessibilité optimale :

- Support des lecteurs d'écran (NVDA, JAWS).
- Compatibilité avec les gestes tactiles adaptés.
- Prise en compte des personnes en situation de handicap.
- Adaptation des couleurs.

4.5.8. ISO/IEC 12207 – Processus de Cycle de Vie du Logiciel

Cette norme définit les phases de développement :

- Conception : Documentation des exigences et architecture logicielle.
- Validation : Tests avant mise en production.
- Maintenance : Suivi des correctifs et évolutions.

4.5.9. ISO/IEC 5055 – Mesure de la Qualité du Code

Pour assurer un code de haute qualité :

- Détection des failles de conception.
- Réduction des redondances et de la complexité cyclomatique.
- Analyse automatique via Gitlab, Trivyou Jenkins.

4.5.10. ISO/IEC 29119 – Tests Logiciels

Cette norme définit une stratégie de tests efficace :

- Tests unitaires et d'intégration.
- Automatisation des tests (CI/CD).
- Tests de performance et de charge.

4.5.11. ISO 14001 – Système de Management Environnemental

Objectif : réduire l'impact écologique du projet :

- Optimisation du code pour limiter la consommation d'énergie.
- Réduction de l'empreinte carbone des infrastructures cloud.

4.6. Pratiques de sécurité

4.6.1. Authentification et contrôle d'accès

- Utilisation de JWT et d'une authentification forte (MFA) sur les interfaces critiques.
- Rôles et permissions configurés selon le principe du moindre privilège.

4.6.2. Chiffrement des données

- Données sensibles chiffrées au repos (AES-256) et en transit (HTTPS/TLS 1.3).
- Gestion des secrets via environnement sécurisé (Vault ou équivalent).

4.6.3. Surveillance et détection

- Journalisation complète des événements critiques (connexion, erreur, faille).
- Intégration d'un monitoring temps réel via Prometheus et Grafana.

Dans le cadre du renforcement de la qualité, nous intégrerons l'outil Trivy au pipeline CI/CD du projet. Il permet de réaliser une analyse statique du code à chaque push, et d'identifier de manière continue :

- Les vulnérabilités de sécurité (injections, fuites potentielles, mauvaises pratiques).
- Les bugs critiques et violations de règles de codage.
- La complexité cyclomatique et la duplicabilité du code.
- La couverture de tests unitaires par fichier ou module.

4.7. Normes de contribution Git

4.7.1. Stratégie de gestion des branches

La stratégie Git adoptée est dérivée du **GitLab Flow**, combinant simplicité et intégration CI/CD continue.

- **Branche principale** (main) : contient les versions stables et déployées en production.
- **Branche de développement** (develop) : intègre les fonctionnalités validées en amont de la production.
- **Branches de fonctionnalités** (feature/*) : issues de develop, elles sont dédiées à l'implémentation de nouvelles fonctionnalités.

- **Branches de correctifs** (fix/*) : issues de develop, elles corrigent des anomalies non critiques.
- **Branches de hotfix** (hotfix/*) : issues de main, utilisées pour les corrections critiques en production.
- **Branches de release** (release/*) : préparent les livraisons intermédiaires.

Format : **<type>/NAAVI-<ticket_id>**

- **type** : nature du travail réalisé.
- **ticket_id** : identifiant unique du ticket correspondant dans GitLab

4.7.2. Convention de nommage des commits

Nous avons adopté une convention de nommage inspirée de la spécification **Conventional Commits**, adaptée aux besoins du projet pour maximiser la traçabilité et la lisibilité dans GitLab.

La convention permet de lier explicitement chaque commit à une branche et un ticket GitLab associé.

Format : **<branch_name> : <description>**

- **branch_name** : correspond au nom exact de la branche Git sur laquelle le commit est effectué.
- **description** : est une phrase courte à l'infinitif décrivant le changement apporté.

4.7.3. Processus de validation des Merge Request (MR)

Le processus de validation des Merge Requests est un pilier fondamental du contrôle qualité. Il garantit que toute contribution passe par un contrôle de qualité rigoureux, favorise la revue croisée et assure la stabilité du code fusionné.

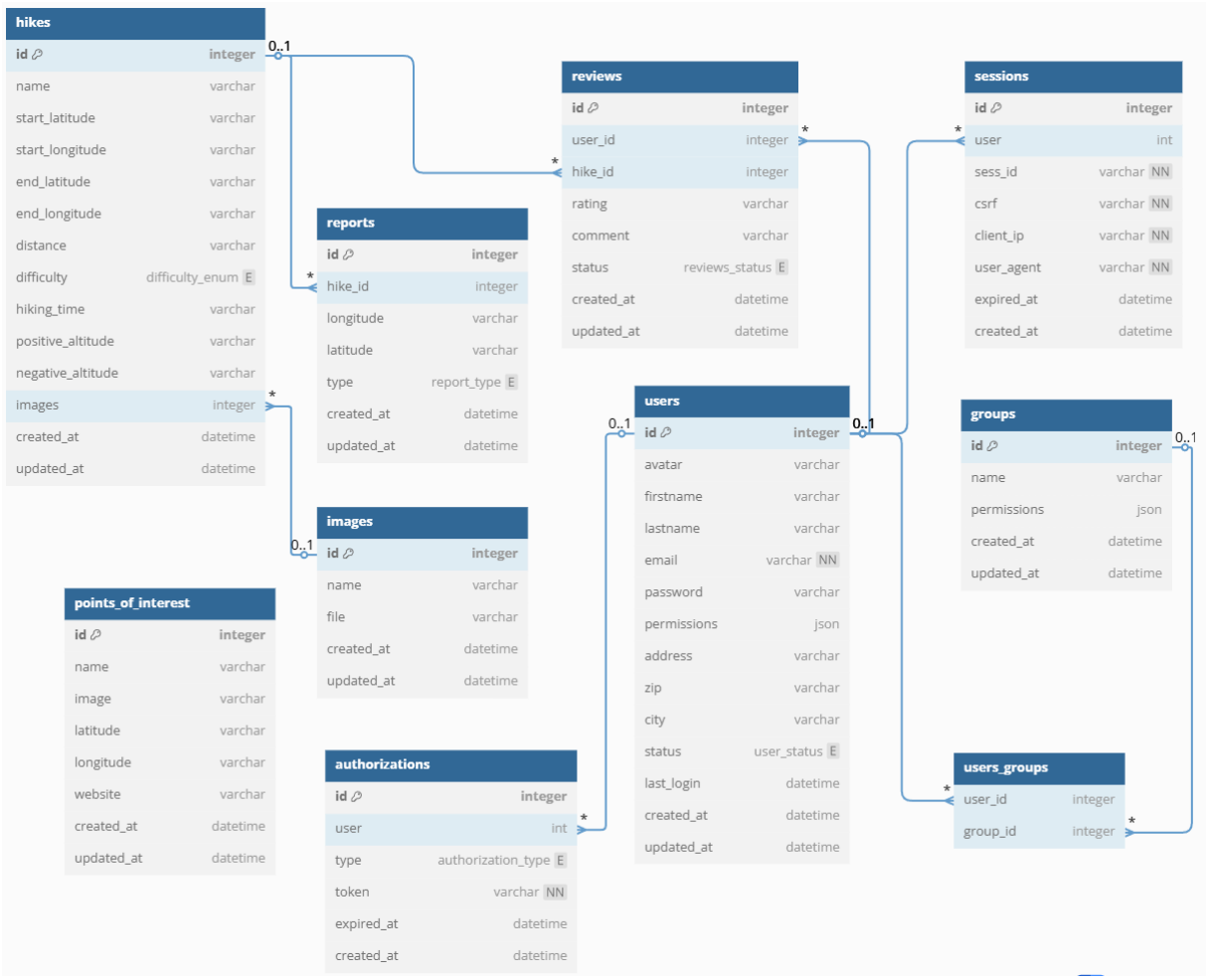
- La MR doit être ouvert à partir d'une branche respectant la convention de nommage.
- Le commit principal doit suivre la convention de nommage du projet.
- La description de la MR doit permettre d'identifier rapidement le ticket lié ainsi qu'une description claire de la contribution apportée.
- La MR doit être assigné au contributeur d'origine.
- Chaque MR déclenche automatiquement un pipeline Gitlab CI, celle-ci permettra d'assurer l'intégrité des développements du contributeur avant

la fusion du code.
Si la pipeline est complet et sans erreur, la MR peut-être fusionner avec la branche de destination.
Si la pipeline n'est pas complet et comporte des erreurs, celles-ci doivent être corrigées par le contributeur ou les relecteurs les anomalies remontées pour relancer une pipeline.

- Une fois la MR fusionnée avec la branche de destination, la branche source est automatiquement supprimée.

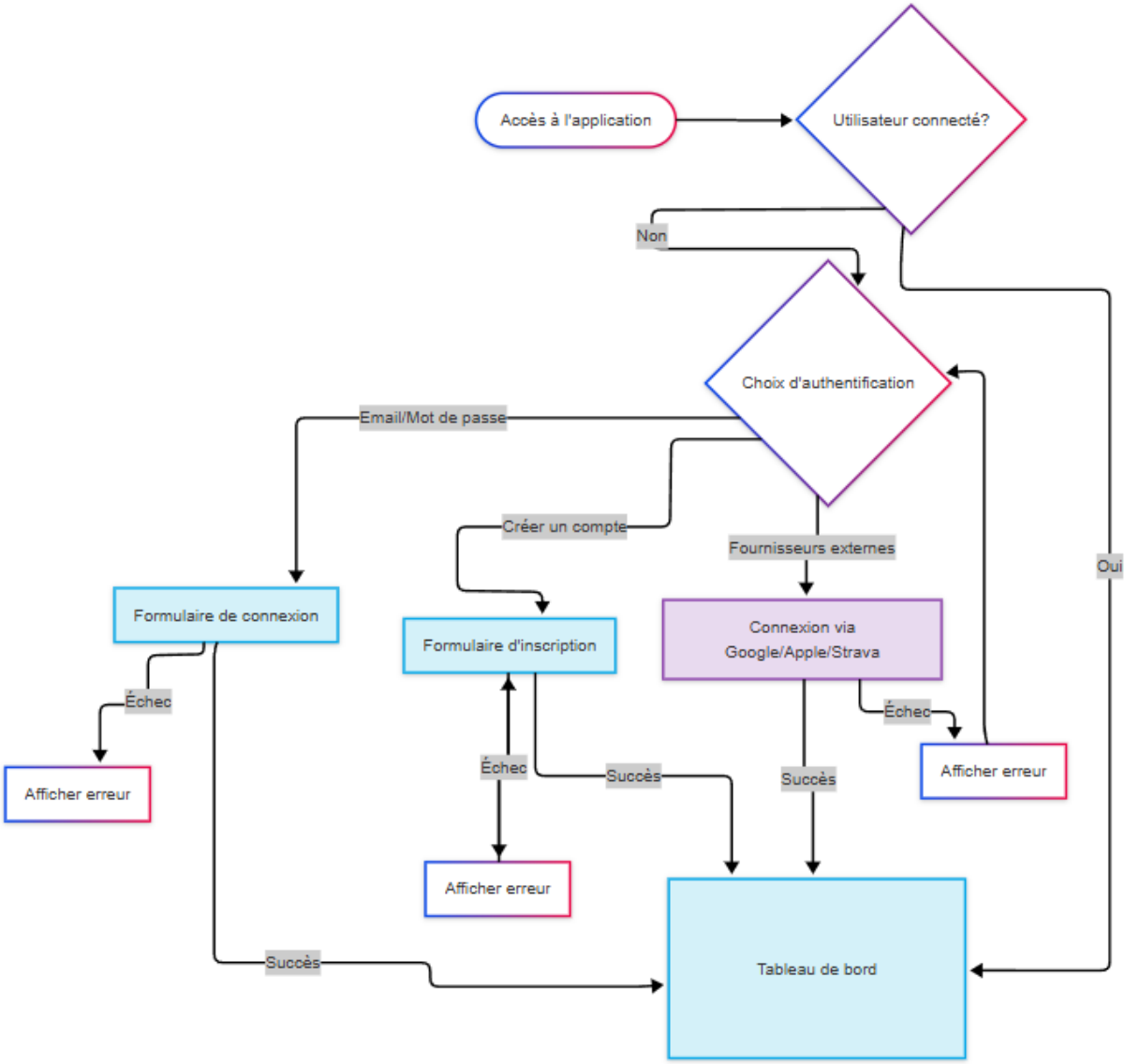
4.8. Conception fonctionnelle

4.8.1. Schéma de base de données

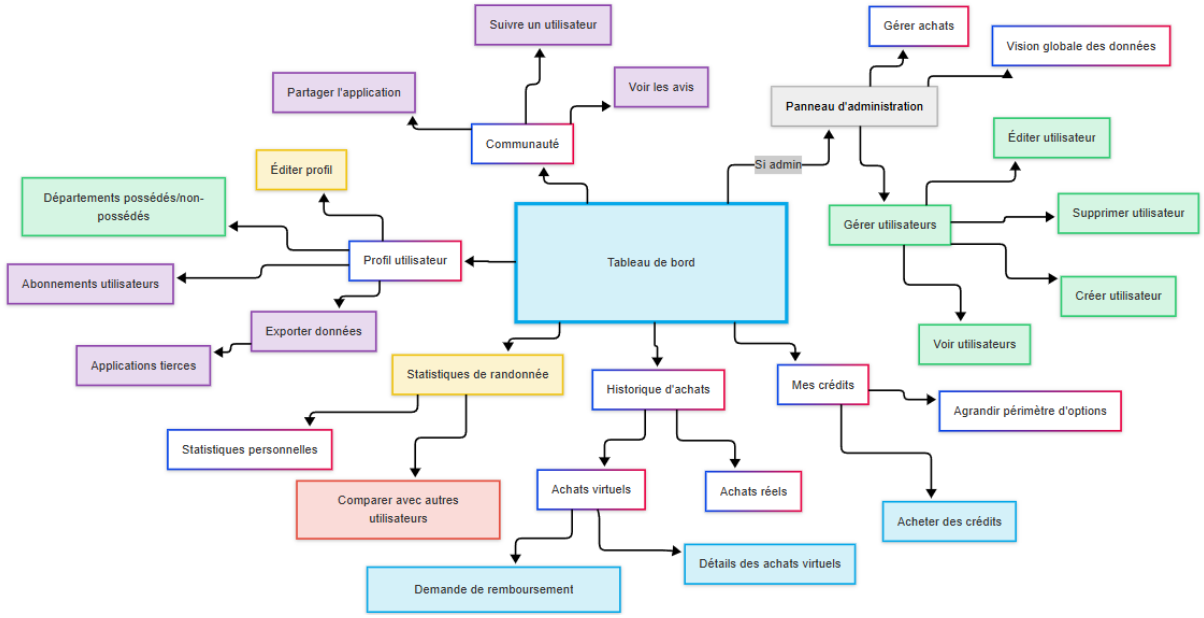


4.8.2. Diagramme de flow

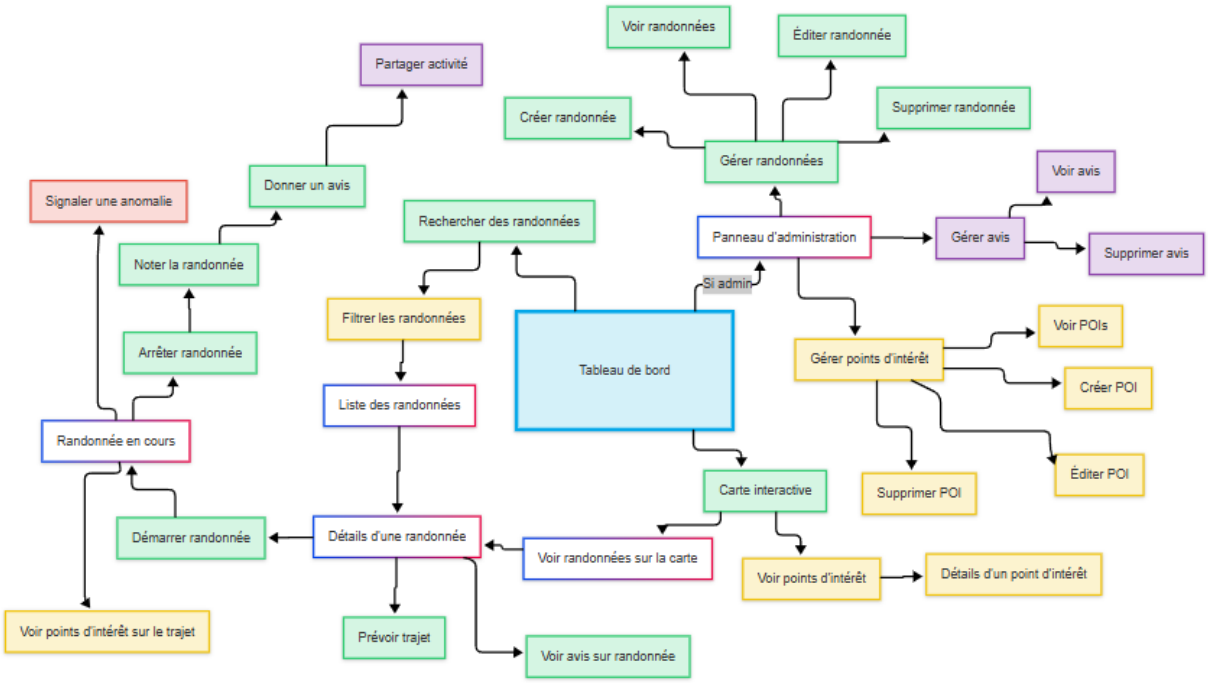
4.8.2.1. Écran de Lancement / Connexion



4.8.2.2. Gestion des utilisateurs



4.8.2.3. Gestion des randonnées



4.8.3. Fonctionnalités de l'application mobile

Les versions (MVP, V1, V2, etc.) sont uniquement des indicateurs de versions pour le bon déroulement du développement de l'application en interne. Elles ne seront pas corrélées avec les versions de développement de l'application.

MVP

- Affichage des premières pages
- Choisir une randonnée et la visualiser sur la carte
- Voir les informations principales de la randonnée sélectionnée
- Lancer une randonnée sélectionnée
- Lancer une randonnée improvisée

V1

- Choisir une randonnée et la visualiser sur la carte
- Chercher une randonnée par secteur
- Créer sa propre randonnée organisée
- Donner mon avis sur la randonnée que je viens d'effectuer
- Visualiser la liste des départements acquis
- Authentification

V2

- Ajouter un point d'intérêt sur mon trajet
- Pré-validé un nouveau point d'intérêt
- Visualiser les POI dans les suggestions de trajets
- Visualiser les POI sur mon trajet
- Éditer les données de mon compte
- Chercher une randonnée par des critères définis
- Visualiser les statistiques de mes randonnées

V3

- Ajouter un signalement sur mon trajet
- Visualiser les signalements des autres utilisateurs
- Pré-valider ou invalider le signalement d'un autre utilisateur
- Permettre de m'identifier avec un type de randonneurs pour des suggestions améliorées
- Visualiser mes statistiques de randonnées

V4

- Partager mon activité sur d'autres applications
- Suivre le compte d'un autre utilisateur
- Partager l'application à un contact

- Authentification par des fournisseurs externes
- Visualiser les avis des autres utilisateurs sur les points d'intérêts
- Ajouter un avis sur un point d'intérêt
- Visualiser les avis des autres utilisateurs sur les randonnées

V5

- Exporter mes données vers d'autre application
- Renseigner une enseigne partenaire à proximité
- Récompenser de l'activité sur l'application
- Laisser un avis sur un partenaire
- Visualiser mon nombre de crédit

4.8.4. Fonctionnalités du back-office

MVP

- Affichage des premières pages
- Authentification

V1

- Gestion des utilisateurs (activation, suspension, suppression)
- Gestion des randonnées (ajout, modification, suppression)

V2

- Gestion des POI (ajout, validation, modification, suppression)

V3

- Gestion des signalements (ajout, validation, modification, suppression)

4.9. Phase de test

4.9.1. Stratégie de test

Dans notre démarche qualité, nous allons définir une stratégie de test rigoureuse afin de garantir la fiabilité et la conformité de nos fonctionnalités avant leur mise en production. Chaque développement doit obligatoirement être validé par au minimum trois intervenants :

- **Le développeur** : responsable de tester et valider son propre travail pour s'assurer que la fonctionnalité développée fonctionne comme attendu.
- **Un second développeur** : chargé de relire le code, de vérifier sa conformité aux bonnes pratiques, et de s'assurer de la maintenabilité et de la qualité technique.
- **Un testeur** : responsable de vérifier que les exigences fonctionnelles et les KPI attendus sont bien respectés à travers des tests dédiés.

Chaque développeur en charge de la création de fonctionnalités devra systématiquement mettre en place différents niveaux de tests :

- **Tests unitaires** : pour valider le bon fonctionnement isolé de chaque composant ou fonction.
- **Tests d'intégration** : pour s'assurer que les différents modules interagissent correctement entre eux.
- **Tests end-to-end (E2E)** : pour vérifier que l'ensemble du parcours utilisateur fonctionne comme prévu dans un environnement proche de la production.

4.9.2. Définition of Done

Une fonctionnalité est considérée comme terminée lorsque :

- Tous les tests unitaires, d'intégration et end-to-end passent avec succès.
- Le code a été relu et validé par un autre développeur.
- La fonctionnalité a été testée par un testeur dédié.
- Le testeur confirme que les KPI attendus sont respectés.
- Le testeur valide que la fonctionnalité est conforme aux maquettes et aux spécifications fonctionnelles.

5. Business Plan

5.1. Objectifs

5.1.1. À court terme (0-3 mois)

- Finaliser le MVP (Sprint 1 et Sprint 2) : authentification sécurisée, gestion CRUD des randonnées, suivi GPS et orientation en direct.
- Atteindre ≥ 70 % de couverture de tests unitaires et d'intégration, maintenir un indice Trviy > 8, assurer une disponibilité ≥ 99 % et un temps de réponse API < 200 ms.
- Recruter et mobiliser 1 000 bêta-testeurs sur Discord/Instagram pour recueillir les premiers retours qualitatifs.
- Établir au moins 3 partenariats avec des offices de tourisme ou fédérations locales pour enrichir les parcours.
- Publier la version MVP sur l'App Store et Google Play d'ici la fin du Sprint 2.

5.1.2. À moyen terme (6-12 mois)

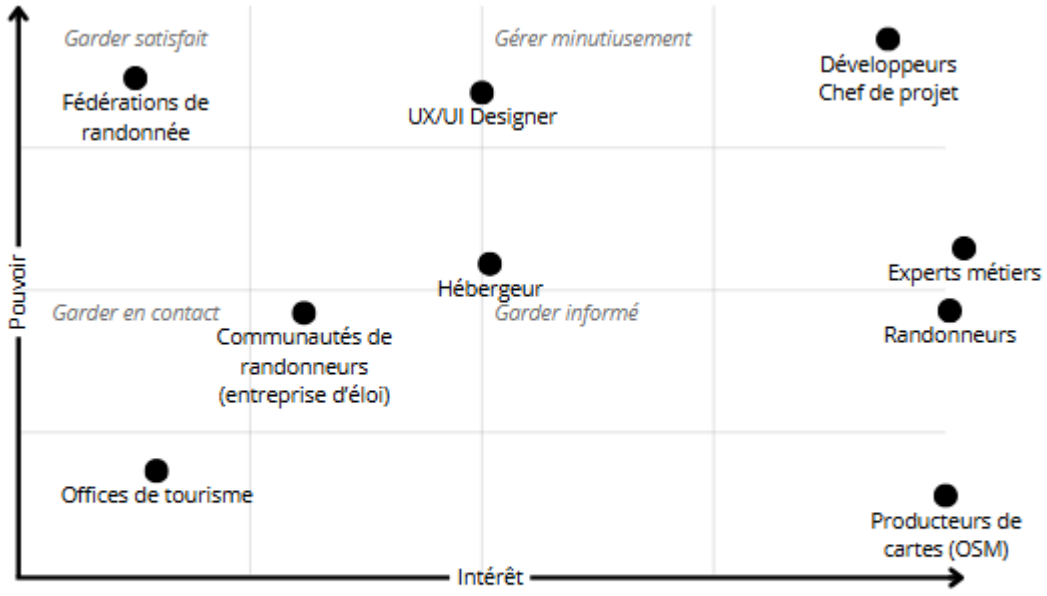
- Livrer V2 & V3 : gestion avancée des POI (CRUD, filtres, clusters) et module de signalement/statistiques en temps réel.
- Déployer V4 (Communauté & Appli tierce) : modération des avis, export vers Strava/Apple Santé, stories Instagram/Snap et fonctionnalités de partage/parrainage.
- Atteindre 50 000 utilisateurs actifs et générer 10 000 achats de zones complémentaires sur les 12 premiers mois.
- Intégrer Apple HealthKit et Google Fit, viser un NPS > 30 et un taux de crash < 1 %.
- Consolider 5 partenariats stratégiques et lancer officiellement le modèle "à la carte" sans abonnement.

5.1.3. À long terme (> 12 mois)

- Finaliser V5 (Partenaire) : gestion complète des partenaires, QR-codes de proximité, notifications pop-up et système de récompenses.
    - Étendre Naavi au web avec un tableau de bord personnel et un mode hors-ligne avancé (téléchargement de cartes et parcours).
- Introduire des défis collectifs géolocalisés, badges personnalisés et autres mécaniques de gamification avancées.

- Atteindre 100 000 utilisateurs actifs, pérenniser le chiffre d'affaires et renforcer la position de Naavi comme référence du marché.
- Maintenir la conformité aux normes ISO et au RGPD, et assurer une maintenance évolutive proactive (hotfix, monitoring et roadmap continue).

5.2. Parties prenantes



Pouvoir : La capacité d'une partie prenante à influencer la prise de décision et la réussite du projet. Plus elle a de pouvoir, plus elle peut **impacter directement les résultats**.

Influence/Intérêt : Le niveau d'intérêt ou d'implication de la partie prenante dans le projet. Plus son intérêt est élevé, plus elle sera **impliquée et impactée par les résultats du projet.**

Développeurs	Acteurs principaux – à impliquer en continu.
Chef de projet	Décideur – communication fréquente et détaillée.
UX/UI Designer	Impliquer régulièrement dans la conception fonctionnelle.
Randonneurs (utilisateurs finaux)	À écouter – importance des retours pour ajuster le produit.
Communautés de randonneurs	Source de diffusion et d'idées – à tenir informées.
Offices de tourisme	Intéressés si l'appli devient un outil local.
Fédérations de randonnée	À consulter – peuvent apporter de la légitimité.
OpenStreetMap	Fournisseurs de ressources – à respecter et suivre leurs conditions.

(producteurs cartes)	
Hébergeur / Fournisseur cloud	Support technique – à surveiller pour la stabilité.
Mentors / Experts métiers	À solliciter ponctuellement – expertise précieuse.

1. Equipe projet :
   - **Développeurs**
     Responsables du développement de l'application web (frontend, backend, base de données, etc.).
   - **Chef de projet**
     Supervise la planification, fixe les priorités et assure la communication avec les autres parties prenantes.
   - **UX/UI Designer**
     Conçoit l'ergonomie et l'expérience utilisateur de l'application.
2. Utilisateurs :
   - **Randonneurs**
     Utilisateurs finaux de l'application, ciblés pour consulter ou partager des parcours de randonnée.
   - **Communautés de randonneurs** (clubs, associations, groupes Facebook/Discord…)
     Cibles potentielles pour la diffusion et la validation des besoins.
     Potentiellement dans l'entreprise d'Eloi
3. Parties prenantes externes :
   - **Partenaires potentiels**
     - Offices de tourisme
     - Fédérations de randonnée
     - Marqueurs de sentiers ou producteurs de contenu cartographique (IGN, OpenStreetMap…)
   - **Hébergeur / Fournisseur de services cloud**
     Hébergeur de l'infrastructure technique (serveurs, base de données, etc.).
   - **Mentors ou experts métiers**
     Professionnels de la randonnée ou de la cartographie pouvant valider la pertinence fonctionnelle.

5.2.1. Early adopters

Comme vu dans le plan de communication, des bêta testeurs seront sélectionnés d'après un questionnaire que les membres de l'équipe auront partagé. Toutes les personnes intéressées pour tester l'appli mais qui ne sont pas bêta testeurs, seront nos premiers utilisateurs

5.2.2. Coachs

Un ou plusieurs coachs accompagnent l'équipe projet tout au long du développement de l'application. Dans notre cas les coachs sont nos chefs de projet, nous avons aussi l'équipe pédagogique qui est là pour nous guider et remplir le rôle de coach. Leur rôle est d'apporter un regard extérieur et bienveillant sur la gestion du projet, l'organisation de l'équipe, les choix techniques ou méthodologiques.
Ils aident à prendre du recul, à débloquer des situations, et à améliorer les pratiques professionnelles des membres de l'équipe.

Les échanges avec les coachs peuvent prendre la forme de points réguliers, de retours sur les livrables ou de moments dédiés à la résolution de problèmes spécifiques.

5.3. Business model

5.4. Stratégie de marketing

5.4.1. Analyse concurrentielle

Nous identifierons les principales fonctionnalités des applications les plus utilisées et les mieux notées sur les stores (App Store & Google Play) : Visorando, AllTrails, Komoot, Strava, Wikiloc et MaRando en analysant leurs forces, faiblesses, positionnement et fonctionnalités clés.

Cette analyse mettra en lumière :

- Le manque de différenciation entre amateurs et professionnels chez la plupart des concurrents.
- L'absence d'un véritable système d'alerte en temps réel sur les dangers ou zones impraticables.
- L'opportunité d'un contenu éditorial et communautaire plus fort (guides locaux, panoramas, lieux remarquables, points de ravitaillement).
- Une expérience souvent uniforme et peu personnalisée selon le profil du randonneur.

Notre proposition de valeur repose sur une expérience de randonnée contextualisée et enrichie, s'appuyant sur la qualité des données de terrain et des points d'intérêt, les retours utilisateurs et la communauté.

5.4.2. Analyse des concurrents principaux

AllTrails :

- Application très populaire à l'international.
- Interface claire, filtres efficaces (distance, difficulté, boucle, etc.).
- Peu de contenu différencié entre débutants et experts.
- Alertes de danger quasi absentes.
- Moins axée sur les points d'intérêt culturels ou atypiques.

Komoot :

- Excellente navigation GPS, carte topographique avancée.
- Bonne recommandation d'itinéraires selon activités (vélo, trail, randonnée).
- Personnalisation selon le profil randonneur limitée.
- Faible aspect communautaire.
- Peu d'intégration de lieux culturels ou historiques.

Visorando :

- Application française, très utilisée dans les régions rurales.

- Itinéraires bien détaillés avec guides papier ou PDF.
- Design ancien, peu moderne.
- Communauté peu visible, peu d'interactions entre utilisateurs.
- Aucune gestion des dangers ou alertes.

Outdooractive :

- Application très complète, utilisée aussi par les pros.
- Collaboration avec les parcs naturels et offices de tourisme.
- Trop complexe pour un usage loisir ou amateur.
- UX peu intuitive, surcharge d'infos.
- Personnalisation très faible de l'expérience utilisateur.

Wikiloc :

- Grande base de données d'itinéraires partagés par les utilisateurs.
- Couverture mondiale.
- Interface datée, ergonomie à améliorer.
- Manque de modération / vérification des contenus.
- Peu d'éléments sociaux ou communautaires.

MaRando :

- Application avec une grande base de données de sentiers français, souvent issus de sources officielles (FFRandonnée).
- Tous les sentiers sont gratuits, contrairement à certains concurrents qui proposent un accès payant aux cartes hors-ligne ou premium.
- Peu d'utilisateurs actifs, ce qui limite les interactions et les retours communautaires.
- Bons avis sur la précision des parcours et la qualité des fiches.
- Faible dimension sociale : pas ou peu de fonctionnalités communautaires (commentaires, signalements, partage de photos, etc.).
- Soutenu par ANS (Agence Nationale du Sport)

5.4.3. SWOT

AllTrails :

- Forces :
  - Base d'utilisateurs massive, interface intuitive, filtres précis.
- Faiblesses :
  - Peu de contenus spécifiques par profil (pro/amateur), peu de données locales sur les dangers.
- Opportunités :

    - Ajout d’éléments communautaires et d’interactions sociales.
- Menaces :
    - Présence dominante sur le marché, forte notoriété.

Komoot :

- Forces :
    - Cartographie précise, bon moteur de recommandations.

  Faiblesses :

    Manque d’aspect social et communautaire, peu orienté sur la découverte de POI naturels ou culturels.
- Opportunités :
    - Développement de communautés locales, contenu immersif.
- Menaces :
    - Intégration avec le matériel GPS pro.

Visorando :

- Forces :
    - Portail francophone de confiance, guides sérieux.
- Faiblesses :
    - Interface datée, manque d’aspect participatif et vivant.
- Opportunités :
    - Modernisation de l’expérience, enrichissement visuel.
- Menaces :
    - Risque de rester sur une niche vieillissante.

Wikiloc :

- Forces :
    - Grande base d’itinéraires partagés par les utilisateurs.
    - Couverture mondiale.
    - Fonctionnalités GPS fiables, même hors-ligne.
- Faiblesses :
    - Interface datée, ergonomie à moderniser.
    - Manque de modération / vérification des contenus.
    - Faible dimension sociale, peu d’interactions entre utilisateurs.
- Opportunités :
    - Mise à jour de l’interface pour séduire une nouvelle génération.
    - Intégration de fonctionnalités communautaires (notes, badges, réactions).
- Menaces :
    - Concurrence accrue d’applications plus modernes et localisées.

    - Baisse de confiance liée à des parcours non vérifiés.

MaRando :

- Forces :
    - Données officielles et fiables sur les sentiers français.
    - Tous les parcours sont gratuits.
    - Bons retours sur la qualité des informations.
    - Renseignement historique fort.
- Faiblesses :
    - Communauté encore peu développée.
    - Manque de contenu visuel immersif (photos, panoramas, vidéos).
- Opportunités :
    - Développement d'un noyau communautaire autour de la randonnée en France.
    - Intégration de services partenaires locaux (hébergements, guides, événements).
- Menaces :
    - Difficile de se démarquer face à des géants internationaux.
    - Ralentissement de l'innovation si le modèle reste trop institutionnel.

5.5. Plan de communication

5.5.1. Développement d'une communauté

La communauté est au cœur du projet, à la fois en tant que producteur de contenu (photos, descriptions, points d'intérêt) et acteur de la sécurité (signalements).

Actions clés :

- Mettre en place des formulaires ciblés afin de recueillir les retours et freins des différents profils utilisateurs (amateurs, professionnels, non-pratiquants), dans une logique d'amélioration continue et de co-construction de l'application
- Lancement d'un groupe pilote avec des randonneurs influents et bêta testeurs (Instagram, Discord) pour tester l'app et partager leurs retours.
- Création d'un serveur Discord autour de la rando, avec canaux par région, par niveau, par centre d'intérêt.
- Création d'un groupe WhatsApp
- Organisation de randonnées "communautaires" pour créer de l'engagement physique autour de la marque.
- Mécaniques de gamification : système de points selon l'implication.

5.5.2. Communication coachs et early adopters

Pour assurer le lancement, il est essentiel d'impliquer des experts dès le départ :

- Ciblage de sportifs / guides via réseaux sociaux comme Instagram, Facebook ou Linkedin .
- Offre ambassadeur : accès gratuit et anticipé à tous les secteurs de randonnées.
- Création d'un réseau de bêta testeurs qui auront pour mission de créer les premiers itinéraires "qualité" sur l'application.